
Security Review Report
NM-1048 - Infrared Router



NETHERMIND
SECURITY

(September 22, 2026)

Contents

1	Executive Summary	2
2	Audited Files	3
3	Summary of Issues	3
4	System Overview	4
4.1	Architecture and Actors	4
4.2	The Swap Lifecycle	4
4.3	Weirroll Program Execution	4
4.4	Multi-Token Swaps and Input Authorization	4
4.5	Administration and Fund Recovery	4
5	Risk Rating Methodology	5
6	Issues	6
6.1	[High] Caller-supplied fee and positive slippage parameters let any user strip protocol fees, partner fees and positive slippage capture from swaps	6
6.2	[Low] A partner can reenter into the proxy while it has funds	8
6.3	[Low] The <code>_execute(...)</code> function misparses revert data that is not an <code>Error(string)</code>	9
6.4	[Low] The output partner fee can make a validated range of <code>outputMin</code> values revert	10
6.5	[Info] Multiswap events sum input amounts across tokens with different decimals	11
6.6	[Info] The <code>executePath(...)</code> function has no access control while it custodies in-flight user funds	12
6.7	[Info] The <code>swapExactInSingle(...)</code> function cannot settle fee-on-transfer tokens	13
6.8	[Info] The <code>swapExactInSingle(...)</code> function credits its entire balance instead of the swap delta	14
6.9	[Info] The locally reproduced Uniswap V4 params struct is pinned to a single Universal Router revision	15
7	Documentation Evaluation	16
8	Test Suite Evaluation	17
8.1	Tests Output	17
8.2	Automated Tools	19
8.2.1	AuditAgent	19
9	About Nethermind	20

1 Executive Summary

This document presents the results of a security review conducted by [Nethermind Security](#) for [Infrared's](#) Router contracts. These contracts implement a swap routing layer in which the trade route is computed off-chain and supplied to the chain as a Weiroll program. The Router holds user token approvals and native ETH, applies a protocol fee and an optional partner fee, and delegates the execution of the route to a separate stateless virtual machine.

The system is designed so that all economic logic resides in the Router while arbitrary execution is confined to the ExecutionProxy contract. The Router determines a trade's output by measuring its own balance before and after execution, optionally retains any amount realized above the quoted figure, and enforces the caller's minimum output before paying the recipient. As part of this engagement, the fix commits ranging from [d9b6dbb](#) through [09cb18b](#) have been reviewed.

The audit comprises 1111 lines of Solidity code. The audit was performed using (a) manual analysis of the codebase, and (b) automated analysis tools.

Along this document, we report 9 points of attention, where one is classified as High, three are classified as Low and five are classified as Informational severity. The issues are summarized in Fig. 1.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 presents the summary of issues. Section 4 presents the system overview. Section 5 discusses the risk rating methodology. Section 6 details the issues. Section 7 discusses the documentation provided by the client for this audit. Section 8 presents the test suite evaluation and automated tools used. Section 9 concludes the document.

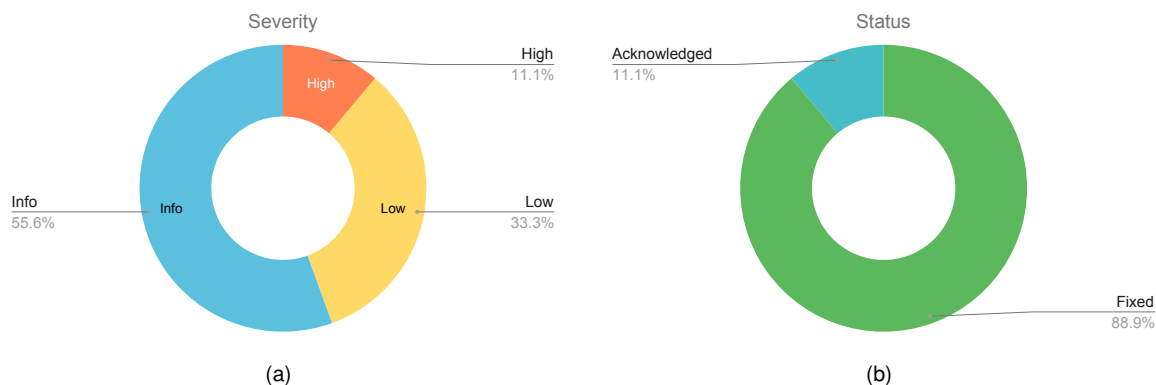


Fig. 1: Distribution of issues: Critical (0), High (1), Medium (0), Low (3), Undetermined (0), Informational (5), Best Practices (0). Distribution of status: Fixed (8), Acknowledged (1), Mitigated (0), Unresolved (0)

Summary of the Audit

Audit Type	Security Review
Initial Report	September 09, 2026
Final Report	September 22, 2026
Initial Commit	275fcd4
Final Commit	09cb18b
Documentation Assessment	Medium
Test Suite Assessment	Medium

2 Audited Files

	File	LoC	Comments	Ratio	Blank	Total
1	contracts/src/ExecutionProxy.sol	11	24	218.2%	5	40
2	contracts/src/Router.sol	551	263	47.7%	97	911
3	contracts/src/weiroll-helpers/Bytes32.sol	6	8	133.3%	1	15
4	contracts/src/weiroll-helpers/SignedMathHelpers.sol	38	67	176.3%	10	115
5	contracts/src/weiroll-helpers/Tupler.sol	8	11	137.5%	1	20
6	contracts/src/weiroll-helpers/BlockchainInfo.sol	15	20	133.3%	4	39
7	contracts/src/weiroll-helpers/ArraysConverter.sol	44	42	95.5%	8	94
8	contracts/src/weiroll-helpers/UniswapV4SwapHelpers.sol	101	151	149.5%	26	278
9	contracts/src/weiroll-helpers/Integer.sol	40	27	67.5%	11	78
10	contracts/src/weiroll-helpers/MathHelpers.sol	87	123	141.4%	28	238
11	contracts/src/weiroll/VM.sol	94	16	17.0%	16	126
12	contracts/src/weiroll/CommandBuilder.sol	112	12	10.7%	18	142
13	contracts/src/interfaces/IExecutor.sol	4	21	525.0%	1	26
	Total	1111	785	70.7%	226	2122

3 Summary of Issues

	Finding	Severity	Update
1	Caller-supplied fee and positive slippage parameters let any user strip protocol fees, partner fees and positive slippage capture from swaps	High	Fixed
2	A partner can reenter into the proxy while it has funds	Low	Fixed
3	The _execute(...) function misparses revert data that is not an Error(string)	Low	Fixed
4	The output partner fee can make a validated range of outputMin values revert	Low	Fixed
5	Multi-swap events sum input amounts across tokens with different decimals	Info	Fixed
6	The executePath(...) function has no access control while it custodies in-flight user funds	Info	Fixed
7	The swapExactInSingle(...) function cannot settle fee-on-transfer tokens	Info	Acknowledged
8	The swapExactInSingle(...) function credits its entire balance instead of the swap delta	Info	Fixed
9	The locally reproduced Uniswap V4 params struct is pinned to a single Universal Router revision	Info	Fixed

4 System Overview

Infrared's execution layer is a swap routing system that lets users trade one token for another along a route computed off-chain. Rather than hardcoding integrations with individual decentralised exchanges, the protocol accepts a small program describing the trade and executes it through a general purpose virtual machine, which keeps the on-chain surface fixed while allowing the off-chain infrastructure to decide the optimal routes. The system is deliberately split in two: a Router that owns every economic decision and holds all user approvals, and a separate executor that performs arbitrary execution but holds no economic logic of its own.

4.1 Architecture and Actors

The deployment consists of three groups of contracts, deployed at deterministic addresses via CREATE3 on each supported chain.

- **Router:** The only contract users interact with directly. It holds their ERC20 approvals and native ETH, applies the protocol and partner fees, measures how much a trade produced, enforces the caller's slippage floor, and pays the recipient.
- **ExecutionProxy:** A Weirroll virtual machine invoked by the Router through `IExecutor.executePath(...)`. It is documented as intentionally stateless, with no storage variables, no owner and no economic logic, so that the arbitrary execution portion of the system stays as small as possible.
- **Weirroll helper contracts:** Stateless contracts that programs call as command targets, providing arithmetic, comparison assertions, type and array manipulation, chain introspection, and a wrapper for Uniswap V4 swaps through the Universal Router.

Four actors participate:

- **User:** Calls one of the four swap entry points, supplying the trade parameters and the program that performs it.
- **Backend quoting engine:** An off-chain service that computes the route, assembles the program, and produces the economic parameters passed to the Router.
- **Owner:** Expected to be a multisig. Rotates the executor, sets the liquidator, and controls the emergency pause.
- **Liquidator:** A wallet authorised alongside the owner to move funds held by the Router so routine sweeps of accrued fees do not require multisig signatures.

4.2 The Swap Lifecycle

After validating the supplied parameters, the Router pulls the input from the caller, measuring the balance it actually received so that tokens delivering less than requested flow through the rest of the swap at their true amount. It then deducts the protocol fee, which simply remains in the contract, and the partner fee, which may be denominated in either the input or the output token. The Router snapshots its balance of the output token, forwards the remaining input to the executor and runs the program, then reads that balance again. The difference between the two readings is treated as the amount the trade produced. Three adjustments follow in a fixed order: any amount above the backend supplied `outputQuote` may be retained by the protocol as captured positive slippage, an output-denominated partner fee is taken if one applies, and the remainder is checked against the caller's `outputMin` before being transferred to the recipient. A single Swap event records both tokens, the input and output amounts, both fee figures and the captured slippage.

4.3 Weirroll Program Execution

The trade is expressed as a Weirroll program: an array of bytes32 commands operating over an array of bytes state slots. Each command packs a function selector, a flags byte, the indices of its input slots, an output slot index and a target address into a single word. The virtual machine walks the commands in order, assembling each call's arguments from the referenced slots and writing results back for later commands to consume. The flags byte selects how the call is made, distinguishing ordinary calls, static calls and calls that attach native ETH, and can mark a command's return data for tuple decoding or extend its argument list into a second word. Among the helper targets, the Uniswap V4 wrapper is the most substantial: it pulls the input token, provides the approvals that Permit2 and the Universal Router require, dispatches a single pool exact input swap, and forwards the realized output to a caller-supplied receiver.

4.4 Multi-Token Swaps and Input Authorization

`swapMulti(...)` generalises the model to several input and several output tokens settled atomically. Validation guarantees that each token appears in exactly one slot across both arrays, which is what makes the per-token balance measurement unambiguous, and allows at most one native slot. Inputs are pulled and staged to the executor first, and each output is then measured, adjusted and paid independently, with one Swap event emitted per output and the aggregate input fees attributed across them. Input authorization takes two forms. The approve version relies on a prior ERC20 approval to the Router. The Permit2 variants instead pull via `permitTransferFrom(...)`, passing `msg.sender` as the signature owner so only the signer can trigger their own authorization, with replay protection enforced by Permit2. Both Permit2 paths reject native ETH; native swaps use the approve path.

4.5 Administration and Fund Recovery

The administrative surface is narrow. The executor is rotated in two steps, proposing an address before promoting it, and the liquidator role can be updated or disabled. The pause gates the four user facing swap entry points, which each carry a reentrancy guard. Two functions allow accrued value to be recovered, both authorised for either the owner or the liquidator: one sweeps token and native balances directly

to a destination, while the other routes the funds through the same fee and slippage pipeline as a user swap, so accumulated fee dust in many tokens can be converted. Both intentionally omit the pause modifier, so funds remain recoverable while the protocol is locked.

5 Risk Rating Methodology

The risk rating methodology used by [Nethermind Security](#) follows the principles established by the [OWASP Foundation](#). The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

- a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;
- b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;
- c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

- a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;
- b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;
- c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Severity Risk		
Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined
		Low	Medium	High
		Likelihood		

To address issues that do not fit a High/Medium/Low severity, [Nethermind Security](#) also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

- a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;
- b) **Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;
- c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

6 Issues

6.1 [High] Caller-supplied fee and positive slippage parameters let any user strip protocol fees, partner fees and positive slippage capture from swaps

File(s): `src/Router.sol#L95-L111`, `src/Router.sol#L276-L286`, `src/Router.sol#L357-L392`

Description: Users obtain a route from the Infrared API (`/v1/quote`, open to anonymous callers) and then call `/v1/build`, which returns calldata for the `swap(...)`, `swapMulti(...)`, `swapPermit2(...)` or `swapMultiPermit2(...)` functions of the Router contract. The Router holds all economic logic: it deducts the protocol fee from the input, pays an optional partner fee, and, when `passPositiveSlippageToUser` is false, keeps any output above `outputQuote` as positive slippage. According to the [documentation](#), the protocol fee is "resolved from your subscription tier (and per-account overrides). Never caller-supplied.", and "the platform protocol fee and positive-slippage routing are derived server-side from the caller's subscription tier". The fee a user pays is therefore the product that paid tiers buy.

However, the Router cannot tell whether the parameters it receives are the ones the backend produced. Every economic field is plain calldata:

```

1 struct SwapParams {
2     // ...
3     // @audit Caller-controlled ceiling above which positive slippage is captured.
4     uint256 outputQuote;
5     // ...
6     // @audit-issue The whole fee and slippage policy is unauthenticated calldata.
7     uint16 protocolFeeBps;
8     uint16 partnerFeeBps;
9     address partnerRecipient;
10    bool partnerFeeOnOutput;
11    bool passPositiveSlippageToUser;
12    // ...
13 }
```

The `_validateSwapCommon(...)` function (and `_validateMultiSwap(...)` for multi-swaps) only enforces upper bounds and a non-zero partner recipient. There is no lower bound, no on-chain fee configuration and no backend signature over the parameters:

```

1 // @audit-issue Only upper bounds are enforced; a zero fee is always accepted.
2 if (p.protocolFeeBps > MAX_PROTOCOL_FEE_BPS) revert ProtocolFeeExceedsCap(p.protocolFeeBps);
3 if (p.partnerFeeBps > MAX_PARTNER_FEE_BPS) revert PartnerFeeExceedsCap(p.partnerFeeBps);
4 if (p.partnerFeeBps > 0 && p.partnerRecipient == address(0)) revert InvalidPartnerRecipient();
```

The `_executeSwap(...)` function (and `_processPulledInputs(...)` with `_settleOutputs(...)` for multi-swaps) then applies whatever the caller supplied:

```

1 function _executeSwap(...) internal returns (uint256 amountOut) {
2     // @audit-issue A zero `protocolFeeBps` or `partnerFeeBps` removes the fee entirely.
3     uint256 protocolFee = (pulled * params.protocolFeeBps) / 10_000;
4     uint256 inputPartnerFee =
5         params.partnerFeeOnOutput ? 0 : (pulled * params.partnerFeeBps) / 10_000;
6     // ...
7     uint256 positiveSlippage;
8     // @audit-issue Setting `passPositiveSlippageToUser` to true, or inflating
9     // `outputQuote`, disables the capture.
10    if (!params.passPositiveSlippageToUser && amountOut > params.outputQuote) {
11        positiveSlippage = amountOut - params.outputQuote;
12        amountOut = params.outputQuote;
13    }
14    // ...
15 }
```

Consequently, a user can build the calldata with a free or low tier key (keys are self-service via POST `/v1/keys`), decode the `SwapParams`, set `protocolFeeBps = 0`, `partnerFeeBps = 0` and `passPositiveSlippageToUser = true`, lower `inputAmount` to the post-fee amount the Weiroll program already consumes, and submit it. They receive the same route that paying subscribers get (multi-hop, split and multi-input/multi-output paths) without paying any fee, while the protocol and any integrator receive nothing. The Swap event reports zero fees, so off-chain accounting cannot flag it, and the permissionless Router cannot enforce the tier afterwards. The Router is already deployed on Ethereum, Base and Arbitrum One.

Note that on-chain checks alone cannot fully close this gap, because the asset being sold is the route and the route is handed to the user in executable form. The `ExecutionProxy` is "stateless and permissionless" by design, so anyone can send their tokens to it and call `executePath(...)` directly with the returned `weirollCommands` and `weirollState`, skipping the Router entirely. Even if the `ExecutionProxy` were restricted to the Router and the Router verified its parameters, the Weiroll program is still disclosed in the calldata, so a user can

deploy their own copy of the VM and run the identical route. On-chain enforcement is therefore a guardrail that raises the effort, not a complete fix, and the residual exposure has to be consciously accepted.

Recommendation(s): Consider deciding explicitly how the tier-based fee model is enforced, since the contracts do not provide the guarantee the documentation describes. The options carry different trade-offs:

- On-chain guardrails. Consider having the backend sign the economic parameters (fee fields, outputQuote, passPositiveSlippageToUser, the hash of the Weiroll program, the taker and an expiry) with a rotatable signer the Router verifies, or moving the fee policy on-chain through an owner-managed protocol fee and registered partner configurations that callers can reference but not lower. This stops the trivial calldata edit but not direct execution through the ExecutionProxy or a redeployed VM, so it should come with explicit acceptance of that residual risk;
- Off-chain gating. Consider serving /v1/build to paying subscribers only, so fees are collected through the subscription and the on-chain fee fields become far less critical, at the cost of free tier reach;
- Relayed execution for the free tier. Consider not returning calldata to free users and submitting the transaction on their behalf so the route never leaves Infrared's infrastructure. The trade-off is that Infrared pays the gas and needs a way to recover it from the user, for example from the swap output, plus the relayer infrastructure;

Whichever option is chosen, consider documenting the resulting trust model for integrators and correcting the public documentation so it no longer states that the protocol fee is never caller-supplied.

Status: Fixed at commit [09cb18b](#)

Update from the client: Fixed in commit [09cb18b](#) (contracts, PR #18). Every swap entry point (swap, swapMulti, swapPermit2, swapMultiPermit2) now takes a backend-signed EIP712 Authorization (nonce, expiry, signature) over the complete parameter struct, the taker (msg.sender), the chain and the Router address (domain InfraredRouter); the fee fields, outputQuote(s), passPositiveSlippageToUser, the partner fields and the keccak of the Weiroll program and state are all inside the signed hash. The _verifyAuthorization runs before validation and before any pull, so a call whose parameters differ from the signed values reverts InvalidAuthorization; unsigned calls are rejected outright (there is no unsigned fallback). Each authorization is single-use (consumedAuthorizations, AuthorizationAlreadyUsed), expiry is capped at MAX_AUTHORIZATION_TTL (1 hour), and the signer is owner-managed via setSigner(newSigner, grace) with a bounded overlap (MAX_SIGNER_GRACE, 1 hour) plus a fail-closed revokeSigner() callable by the owner or the liquidator. Covered by Router.Authorization.t.sol: with verification stubbed out, 63 tests fail (zero fees, passPositiveSlippageToUser flips, inflated outputQuote, tampered program, wrong taker, cross-chain and cross-Router replay, reuse, expiry, signer rotation and revocation) and every other suite passes unchanged. The backend half (gateway signs on /v1/build, never emits unsigned calldata, and the Go hasher is checked for parity against the Router's own hashSwapAuthorization/hashMultiSwapAuthorization views on the merged bytecode) is complete and reviewed as infrared PR #773 (commit [82011677](#)); it is intentionally held unmerged until the new Router is deployed, because merging deploys the gateway to production and the live Routers still accept unsigned calldata. The residual risk is accepted as stated in the Questions section: the program remains disclosed in calldata and can be replayed through a self-deployed VM, and the signer is a backend hot key whose compromise is bounded by rotation. The public documentation stating the protocol fee is "never caller-supplied" is corrected in the same backend change. Not yet performed: Router redeploy (salt bump, per-chain signer), retirement of the current Routers, and the first-option trust-model note for integrators, which will accompany the deployment.

6.2 [Low] A partner can reenter into the proxy while it has funds

File(s): `src/Router.sol`, `src/ExecutionProxy.sol`

Description: The `swapMulti(...)` function lets a user swap several input tokens in one atomic transaction. Third party integrators can be credited a referral fee on those swaps by being set as the `partnerRecipient`, and the fee is paid out of each input token as that input is processed. Partners are external integrators rather than a protocol defined trusted role: the Router pays them a commission, but it does not entrust them with user funds.

Inputs are prepared one at a time inside a loop in the `_processPulledInputs(...)` function. For each token the partner fee is paid out first, and the remainder is then staged to the executor:

```

1  for (uint256 i = 0; i < n; ++i) {
2      address token = params.inputTokens[i];
3      // ...
4      if (inputPartnerFee > 0) {
5          // @audit-issue The partner receives control here, before this slot is
6          // staged and after every earlier slot already has been.
7          _transferOut(token, params.partnerRecipient, inputPartnerFee);
8      }
9      // ...
10     if (token == NATIVE_ETH_SENTINEL) {
11         nativeForwardAmount = forward;
12         effectiveInputToken = NATIVE_ETH_SENTINEL;
13     } else {
14         IERC20(token).safeTransfer(exec, forward);
15     }
16 }

```

For a native ETH slot the payout is performed by `_transferOut(...)` using a low level call with no gas limit, so the `partnerRecipient` contract can reenter:

```

1  if (token == NATIVE_ETH_SENTINEL) {
2      // @audit-issue Full gas is forwarded to a caller-supplied address.
3      (bool ok, ) = to.call{value: amount}("");
4      if (!ok) revert ETHTransferFailed();
5  }

```

The native slot is not required to be the first one. The `_validateMultiSwap(...)` function only scans for the first native entry in order to reconcile `msg.value`, so an input array such as `[USDC, USDT, NATIVE]` is a valid configuration. On the third iteration the partner can execute arbitrary code at a moment when the first two slots' tokens are already sitting in the executor.

The `nonReentrant` modifier on `swapMulti(...)` does not protect those funds. It guards the Router's own state, but the tokens are held by the Executor contract at this point, and the `executePath(...)` function on that contract has no restrictions and no reentrancy guard of its own. From the callback the partner simply calls `executePath(...)` with a program that transfers the tokens to themselves. The executor's documentation justifies having no guard on the premise that "the Router enforces the `nonReentrant` boundary", which does not hold for this window.

This means a malicious partner can take a portion of a victim's input tokens mid transaction, assuming the resulting shortfall stays within the victim's slippage tolerance. `_settleOutputs(...)` still clears every `outputMins[j]`, the swap completes normally, and the emitted Swap event reports a successful trade, so the theft leaves no visible trace. Extraction per swap is bounded by the difference between the realized output and `outputMin`, and it is repeatable across every user the partner routes.

Recommendation(s): Consider adding an "onlyRouter" modifier on the Executor contract. Restricting `executePath(...)` to the Router contract which implements `nonReentrant` already, prevents this attack.

Status: Fixed at commit [6400441](#)

Update from the client: Fixed in commit [6400441](#) (same change as the access-control finding). With `executePath` Router-only, a `partnerRecipient` that gains control during the native-ETH fee payout can no longer run a program against the inputs already staged on the executor; every execution sits behind the Router's `nonReentrant` boundary. Reproduced with a `ReentrantPartner` mock reentering from `receive()` during `swapMulti` with inputs `[tokenA, NATIVE]`: with the gate removed the theft succeeds and the swap still settles; with it the reentry reverts `NotRouter`, the staged balance is intact and the swap completes.

6.3 [Low] The `_execute(...)` function misparses revert data that is not an `Error(string)`

File(s): `src/weiroll/VM.sol`

Description: The Weiroll virtual machine runs each command in a program and, when a sub call fails, unwraps the failure reason so it can be surfaced in a typed `ExecutionFailed` error. The unwrapping assumes the sub call reverted with a string error, whose encoding places the string's length at byte offset 36 of the returndata.

The only guard applied before that assumption is a non-empty check:

```

1  if (!success) {
2      // @audit-issue Only emptiness is checked, not the shape of the payload.
3      if (outdata.length > 0) {
4          assembly {
5              outdata := add(outdata, 68)
6          }
7      }
8      revert ExecutionFailed({
9          // @audit-issue The failing command index is hardcoded although `i` is in scope.
10         command_index: 0,
11         target: address(uint160(uint256(command))),
12         message: outdata.length > 0 ? string(outdata) : "Unknown"
13     });
14 }
```

Advancing the pointer by 68 bytes moves the length word to what was byte offset 36 of the returndata. For an `Error(string)` payload that offset holds the string length. For a custom error carrying two words it holds the error's second argument, which is then treated as a byte length and copied when the message is encoded. The protocol's own `InsufficientOutputAmount(uint256 amountOut, uint256 minAmountOut)` error has that shape, so a slippage failure inside a Weiroll path causes `minAmountOut` to be read as a memory length. Payloads shorter than 68 bytes are read past their allocation instead, returning zero and collapsing the message to "Unknown".

The impact is twofold. A transaction failing inside a Weiroll path consumes more gas than intended and reverting corrupts the data.

Recommendation(s): Consider verifying both that the payload is long enough and that its selector corresponds to `Error(string)` before advancing the pointer, and bubbling the raw returndata unchanged otherwise. The loop index should also be reported instead of the hardcoded zero.

Status: Fixed at commit [030d0d4](#)

Update from the client: Fixed in commit [030d0d4](#). The VM now classifies revert data before touching it: only a provably well-formed `Error(string)` (selector, 68-byte header, offset `0x20`, length within bounds) is unwrapped into `ExecutionFailed`, empty data maps to "Unknown", and every other payload (custom errors, `Panic`, malformed data) is re-raised byte-for-byte. The reported index is the position of the failing command's first word in `commands[]`. Covered by a test suite built from Nethermind's PoC scenarios and gas harness with the assertions inverted to the fixed behaviour (raw returndata checked byte-for-byte under a fixed gas cap, including the 1e18 second-word case, the three-word leak, and consumption across allowances); 10 of the 11 fail on the old VM and all pass on the fix. `ExecutionProxy` salt namespace rotated to `.v4` for redeploy.

6.4 [Low] The output partner fee can make a validated range of outputMin values revert

File(s): [src/Router.sol](#)

Description: A swap carries two output limits. The outputQuote value is the amount the backend quoted, and the outputMin value is the absolute minimum amount the user must be credited. Two separate features then reduce the amount the executor delivered before that minimum is checked. When passPositiveSlippageToUser is false, anything above outputQuote is retained by the protocol and the user is credited exactly outputQuote. When partnerFeeOnOutput is true, the partner fee is denominated in the output token and deducted from what the user is credited.

The validation accepts any outputMin up to and including outputQuote:

```
1 // @audit Equality is permitted, so `outputMin == outputQuote` is a valid input.
2 if (p.outputMin > p.outputQuote) revert InvalidSlippageBounds();
```

The execution path then caps the amount, deducts the fee from the capped value, and only afterwards compares against the minimum:

```
1 uint256 positiveSlippage;
2 if (!params.passPositiveSlippageToUser && amountOut > params.outputQuote) {
3     positiveSlippage = amountOut - params.outputQuote;
4     // @audit The surplus is retained here, so the fee below is charged on
5     // `outputQuote` rather than on the amount the executor delivered.
6     amountOut = params.outputQuote;
7 }
8
9 uint256 outputPartnerFee;
10 if (params.partnerFeeOnOutput && params.partnerFeeBps > 0) {
11     outputPartnerFee = (amountOut * params.partnerFeeBps) / 10_000;
12     // @audit-issue The fee is subtracted from the capped amount.
13     amountOut -= outputPartnerFee;
14     _transferOut(params.outputToken, params.partnerRecipient, outputPartnerFee);
15 }
16
17 // @audit-issue The minimum is checked last, against the twice-reduced amount.
18 if (amountOut < params.outputMin) {
19     revert SlippageExceeded(params.outputToken, amountOut, params.outputMin);
20 }
```

Because amountOut is capped at outputQuote once the cap engages, the most the user can ever be credited is outputQuote minus the partner fee. Validation, however, allows outputMin all the way up to outputQuote. Every outputMin above the reduced amount therefore reverts no matter how well the trade executed.

This affects any swap configured with partnerFeeOnOutput set to true, a non-zero partnerFeeBps, and passPositiveSlippageToUser set to false. With an outputQuote of 1000e18 and a partnerFeeBps of 200, the most the user can be credited is 980e18, so every outputMin above 980e18 is unreachable. A one percent tolerance places outputMin at 990e18, already inside that range, so the swap cannot succeed.

This means that there are cases in which a parameter combination the validation logic accepts as valid, can never settle in reality, and the user pays gas for a transaction that could not have succeeded.

Recommendation(s): Consider enforcing the outputMin floor against the amount before the output-side partner fee is deducted, or alternatively tightening the validator so that it rejects an outputMin that the configured partnerFeeBps makes unreachable. Whichever approach is taken should be mirrored in _settleOutputs(...).

Status: Fixed at commit [8c64a74](#)

Update from the client: Fixed in commit 8c64a74 by tightening the validator (second recommended option). outputQuote stays the gross ceiling and outputMin the net floor, which is how the backend already sizes them; _validateSwapCommon and _validateMultiSwap now compute the reachable ceiling outputQuote - fee(outputQuote) (capture on, output-side fee) with the same flooring as settlement and revert OutputMinUnreachable(outputMin, maxReachable) before pulling any input. Pass-through or no output fee leaves behaviour unchanged. The previous test encoded the flagged case as a post-execution SlippageExceeded; it is replaced by validation, ceiling, short-swap and pass-through tests plus a multi-swap mirror, and the short-payment fuzz oracle now covers the ceiling.

6.5 [Info] Multiswap events sum input amounts across tokens with different decimals

File(s): [src/Router.sol](#)

Description: The `swapMulti(...)` function accepts several input tokens and several output tokens in one transaction, and emits one Swap event per output. The event carries a single input token and a single input amount, and its documentation states that the ten fields are sufficient to reconstruct full fee attribution off chain.

The values placed in those two fields are produced by accumulating across every input:

```
1  uint256 n = params.inputTokens.length;
2  // @audit-issue One arbitrary token is chosen to label the aggregate.
3  effectiveInputToken = params.inputTokens[0];
4  address exec = executor;
5  for (uint256 i = 0; i < n; ++i) {
6      address token = params.inputTokens[i];
7      // @audit-issue Amounts of different tokens are summed into one scalar.
8      inputAmountSum += params.inputAmounts[i];
9      // ...
10     if (token == NATIVE_ETH_SENTINEL) {
11         nativeForwardAmount = forward;
12         effectiveInputToken = NATIVE_ETH_SENTINEL;
13     }
14 }
```

Adding an amount denominated in a six decimal token to one denominated in an eighteen decimal token produces a wrong number and labelling it with whichever token occupies the first slot makes it misleading. For inputs of `10_000e6` USDC and `1e18` WETH the emitted amount is `1000000010000000000` tagged as USDC, and WETH is absent from every event the swap produces.

This means the documented reconstruction does not hold for multi input swap. The failure is silent because the ratio between the aggregated fee and the aggregated amount still approximates the configured basis points, so a check on the implied rate passes while every consumer that reads the amounts as values is wrong by the decimal difference between the tokens. Any off chain process that relies on these numbers will be wrong.

Recommendation(s): Consider emitting a dedicated multi swap event that carries the input token and amount arrays and compute the amounts accordingly.

Status: Fixed at commit [54435a9](#)

Update from the client: Fixed in commit `54435a9`. Multi-swaps now emit a single `MultiSwap` event carrying parallel per-token arrays (input tokens and amounts, per-input protocol and partner fees, output tokens, gross and net outputs, per-output partner fees and captured slippage). Nothing is summed or split across tokens, and Swap is emitted only by the single-swap entry points. The pro-rata splitting helpers were removed. Covered by a mixed-decimals test (`10_000e6 + 1e18` inputs with fees and captured slippage) that checks every emitted value against on-chain balances.

6.6 [Info] The `executePath(...)` function has no access control while it custodies in-flight user funds

File(s): `src/ExecutionProxy.sol`, `src/Router.sol`

Description: The `ExecutionProxy` contract is the Weiroll virtual machine that the `Router` contract delegates trade execution to. During a swap the `Router` transfers the user's input tokens to the executor and then asks it to run the trade program. The executor is documented as intentionally stateless and permissionless, holding no approvals and no economic logic.

The `executePath(...)` function that runs the program is callable by anyone:

```
1 // @audit-issue There is no caller restriction, no owner and no reentrancy guard.
2 function executePath(bytes32[] calldata commands, bytes[] calldata state)
3     external
4     payable
5     override
6 {
7     _execute(commands, state);
8 }
```

The Weiroll virtual machine supports calls to arbitrary targets with arbitrary calldata and arbitrary native value, so `executePath(...)` is a general-purpose "make this contract do anything" primitive available to every address. The contract has no owner and no sweep function, and the `receive()` and `fallback()` functions are both payable.

At the same time, the `Router` hands custody of user funds to that contract inside `_forwardToExecutor(...)`:

```
1 IERC20(token).safeTransfer(exec, amount);
2 IExecutor(exec).executePath(commands, state);
```

This means that any token or native balance the executor holds outside an in-flight swap can be taken by anyone using a two command program that reads `balanceOf(...)` and then calls `transfer(...)`.

The `Router` forwards an amount to a program whose amounts are encoded off-chain before the transaction runs, and it never checks that the executor's balance returned to zero afterwards. The only post condition it enforces is that the output amount is $\geq$ `outputMin`.

Recommendation(s): Consider storing the `Router` address in the executor at construction time and restricting `executePath(...)` so that only the `Router` can invoke it. This ensures that the only entrypoint into the virtual machine is the `Router` contract.

Status: Fixed at commit [6400441](#)

Update from the client: Fixed in commit [6400441](#). `ExecutionProxy` now takes the `Router` in its constructor (immutable `ROUTER`) and `executePath` reverts `NotRouter(caller)` for any other sender, so only the `Router` can drive the VM and any balance parked on the executor is unreachable to third parties. `receive()/fallback()` stay payable for `Router`-forwarded ETH and WETH unwraps. Deploy script and verification updated for the constructor argument. Covered by tests for a third-party caller, a third-party sweep of parked funds, and the same program succeeding when the `Router` drives it. Same commit resolves the partner-reentrancy finding.

6.7 [Info] The swapExactInSingle(...) function cannot settle fee-on-transfer tokens

File(s): `src/weiroll-helpers/UniswapV4SwapHelpers.sol`, `src/Router.sol`

Description: The `_pullInput(...)` function measures the balance difference before and after a token transfer instead of relying on the declared input amount. However, the `_processPulledInputs(...)` function doesn't do the same.

```

1  function _processPulledInputs(...) internal returns (...)
2  {
3  //..
4      //@audit-issue inputAmounts is the gross amount before any FoT fees are applied
5      inputAmountSum += params.inputAmounts[i];
6
7      //@audit pulled already has some fees subtracted due to the transfer
8      //that occurred inside _pullInput
9      uint256 pulled = pulledArr[i];
10 //..
11
12     totalProtocolFees += protocolFee;
13     totalInputPartnerFees += inputPartnerFee;
14     uint256 forward = pulled - protocolFee - inputPartnerFee;
15
16 //..
17     } else {
18         //@audit-issue for a FoT tokens another fee is applied here. The executor receives `forward - fees`
19         IERC20(token).safeTransfer(exec, forward);
20     }
21 }
22

```

The swap is an exact input trade for `amountIn`, Uniswap V4 opens a debt for the specified `amountIn`. The `SETTLE_ALL` action reads that debt, confirms it does not exceed the supplied maximum, and settles it by having Permit2 pull funds from the helper. There are two places where this can revert, depending on whether the helper happens to hold enough token balance:

- **The helper holds only what arrived.** The Permit2 pull asks the token to move `amountIn` out of a contract holding `amountIn` minus the transfer fee, so the token's own `transferFrom` fails. The transaction reverts inside the Permit2 transfer;
- **The helper holds enough to cover the shortfall.** The pull succeeds, but the transfer into the pool manager will withhold another fee. Uniswap V4 credits only what arrived and the `_settle(...)` function computes `paid` as the difference between its synced reserves and its balance afterwards. The amount transferred is not enough to cover the debt and Uniswap reverts with `CurrencyNotSettled`;

Recommendation(s): The protocol would have to precompute all the fees offchain and build the weiroll command in a way that accounts for all the transfers and their fees in ordered for the swap to succeed.

Status: Acknowledged

Update from the client: Won't fix. Fee-on-transfer tokens are unsupported by decision. Natspec that implied support (Router `_pullInput` / `_pullInputViaPermit2`, FR-15) was corrected and the same statement added to `UniswapV4SwapHelpers` in commit `d9b6dbb`. No logic changes.

6.8 [Info] The swapExactInSingle(...) function credits its entire balance instead of the swap delta

File(s): [src/weiroll-helpers/UniswapV4SwapHelpers.sol](#)

Description: The UniswapV4SwapHelpers contract is a Weiroll command target that performs a single-pool Uniswap V4 exact-input swap through the Universal Router and returns the realized output amount. It is deployed once per chain as a CREATE3 singleton shared by every user of the protocol, the swapExactInSingle(...) function is public payable with no access control, the receiver of the proceeds is a caller-supplied parameter, and the receive() function accepts native ETH from anybody.

After the Universal Router call returns, the function determines the swap's output by reading its own total balance rather than the difference across the call:

```

1  UNIVERSAL_ROUTER.execute{value: msg.value}(commands, inputs, deadline);
2
3  if (currencyOut == address(0)) {
4      // @audit-issue This is an absolute balance read, not a pre/post delta.
5      amountOut = address(this).balance;
6  } else {
7      // @audit-issue This is an absolute balance read, not a pre/post delta.
8      amountOut = IERC20(currencyOut).balanceOf(address(this));
9  }
10 if (amountOut < minAmountOut) {
11     revert InsufficientOutputAmount(amountOut, minAmountOut);
12 }

```

There is no pre-call snapshot anywhere in the function, so whatever the contract already held is counted as this caller's output. The entire amount is then paid out to the caller-chosen recipient:

```

1  if (currencyOut == address(0)) {
2      (bool ok, ) = receiver.call{value: amountOut}("");
3      if (!ok) revert InvalidValue();
4  } else {
5      // @audit-issue The full balance, not the swap's output, is sent to `receiver`.
6      IERC20(currencyOut).safeTransfer(receiver, amountOut);
7  }

```

This means that any leftover balance left in the contract will go to whoever calls the function next.

The same issue also corrupts the value the contract returns. The returned amountOut will be inflated by whatever leftover balance the contract has, so a Weiroll command that uses the return value into a downstream step will request more tokens than the swap produced. This contradicts the contract's own documentation, which states that "This contract holds no persistent state — tokens flow through the same call frame they entered in". This condition is not enforced anywhere in the code.

Recommendation(s): Consider snapshotting the balance of currencyOut (and of native ETH) before the Universal Router call and deriving amountOut from the difference between the post-call and pre-call balances, so that only the value the swap actually produced is credited and paid out.

Status: Fixed at commit [5467a92](#)

Update from the client: Fixed in commit 5467a92. swapExactInSingle now snapshots its tokenOut balance before the Universal Router call and credits only the post/pre difference, so a pre-existing balance is neither returned as amountOut nor paid to receiver, and minAmountOut is enforced on the delta. VERSION bumped to 3 and the CREATE3 salt rotated for redeploy. Reproduced by new unit tests against a mock Universal Router and a mainnet fork test that pre-funds the helper.

6.9 [Info] The locally reproduced Uniswap V4 params struct is pinned to a single Universal Router revision

File(s): [src/weiroll-helpers/UniswapV4SwapHelpers.sol](#)

Description: To avoid pulling in the full Uniswap V4 dependency trees, the helper reproduces the periphery's exact input parameter struct locally and encodes it. The contract's own documentation acknowledges that this makes the field ordering load bearing, noting that the struct shape "must match periphery byte-for-byte".

```
1 struct ExactInputSingleParams {
2     PoolKey poolKey;
3     bool zeroForOne;
4     uint128 amountIn;
5     uint128 amountOutMinimum;
6     // @audit This field's presence and position are pinned to one periphery revision.
7     uint256 minHopPriceX36;
8     bytes hookData;
9 }
```

Against a periphery version that does not match the layout of the struct, the word is consumed by whatever occupies that position and the decode of the trailing dynamic argument shifts accordingly. This will lead to reverting swaps.

Uniswap's Universal Router address differs per chain, and some chains hold two versions of the Universal Router contract. You can find a list with the deployed instances [here](#). The "legacy" version has an `ExactInputSingleParams` struct with only 5 fields instead of 6 and on some chains, both UR versions are deployed.

Recommendation(s): Before deploying the protocol consider checking the layout of the `ExactInputSingleParams` struct for each Universal Router contract, for each chain, in order to ensure compatibility. Some chains differentiate between the two versions, while others don't and they simply return a single UR contract address without specifying the version.

Status: Fixed at commit [51c7175](#)

Update from the client: Fixed in commit [51c7175](#). Per-chain fork tests (`test_Layout_Ethereum / Base / ArbitrumOne`) deploy the helper with the exact Universal Router the deploy script pins and run a real USDC -> USDT swap; all three pass against the pinned 2.1.1 routers. `./deploy.sh deploy|dry-run` now runs the target chain's layout test as a gate and refuses to proceed if it fails or cannot run. The guard uses an ERC20/ERC20 pool deliberately: with the legacy 5-field router, native ETH pools succeed by accident (the misread `hookData` offset lands on `currency0 == address(0)`), while USDC -> USDT reverts, which we verified against the legacy mainnet router. Adding a chain now requires a matching layout test.

7 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Smart contracts can use various types of software documentation. Some of the most common types include:

- Technical whitepaper: A technical whitepaper is a comprehensive document describing the smart contract's design and technical details. It includes information about the purpose of the contract, its architecture, its components, and how they interact with each other;
- User manual: A user manual is a document that provides information about how to use the smart contract. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities of the contract;
- Code documentation: Code documentation is a document that provides details about the code of the smart contract. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- API documentation: API documentation is a document that provides information about the API (Application Programming Interface) of the smart contract. It includes details about the methods, parameters, and responses that can be used to interact with the contract;
- Testing documentation: Testing documentation is a document that provides information about how the smart contract was tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- Audit documentation: Audit documentation includes reports, notes, and other materials related to the security audit of the smart contract. This type of documentation is critical in ensuring that the smart contract is secure and free from vulnerabilities.

These types of documentation are essential for smart contract development and maintenance. They help ensure that the contract is properly designed, implemented, and tested, and they provide a reference for developers who need to modify or maintain the contract in the future.

Remarks about Infrared's documentation

Infrared's team has provided written documentation that can be seen [here](#). The docs explain what Infrared is and how their APIs work.

The repository also included two additional files, an `docs/evm-deployments.md` and a `PRODUCTION_PUSH_PLAN.md` file. These files provide more context about how Infrared plans to handle the protocol's deployment and some more context about the current status of the project.

Throughout the engagement, Infrared's team responded to Nethermind's security team's questions during sync calls and async communication channels.

8 Test Suite Evaluation

8.1 Tests Output

```
>forge test

Ran 1 test for test/weirroll-helpers/UniswapV4SwapHelpers.fork.t.sol:UniswapV4SwapHelpersForkTest
[FAIL: EvmError: Revert] setUp() (gas: 0)
Suite result: FAILED. 0 passed; 1 failed; 0 skipped; finished in 368.90µs (0.00ns CPU time)

Ran 22 tests for test/weirroll-helpers/MathHelpers.t.sol:MathHelpersTest
[PASS] test_AddOverflowReverts() (gas: 9098)
[PASS] test_AddSubRoundTrip() (gas: 7883)
[PASS] test_Conditional_AppliesWhenTrue() (gas: 7252)
[PASS] test_Conditional_SkipsWhenFalse() (gas: 6106)
[PASS] test_DivByZeroReverts() (gas: 9172)
[PASS] test_MinMaxAverage() (gas: 8540)
[PASS] test_ModByZeroReverts() (gas: 8846)
[PASS] test_MulDivRoundTrip() (gas: 7795)
[PASS] test_MulDivUp_DenominatorOneIdentity() (gas: 5988)
[PASS] test_MulDivUp_DenominatorZeroReverts() (gas: 8777)
[PASS] test_MulDivUp_NoRoundWhenExact() (gas: 7153)
[PASS] test_MulDivUp_OverflowOnRoundUp() (gas: 9313)
[PASS] test_MulDivUp_RoundsUpWhenRemainder() (gas: 7108)
[PASS] test_MulDiv_BpsFloor() (gas: 6416)
[PASS] test_MulDiv_DenominatorOneIdentity() (gas: 5888)
[PASS] test_MulDiv_DenominatorZeroReverts() (gas: 8773)
[PASS] test_MulDiv_MaxOverMax() (gas: 6283)
[PASS] test_MulDiv_ProductOverflowsButResultFits() (gas: 6544)
[PASS] test_MulDiv_ResultOverflowReverts() (gas: 9417)
[PASS] test_SubUnderflowReverts() (gas: 8720)
[PASS] test_Sum() (gas: 7467)
[PASS] test_Version() (gas: 5705)
Suite result: ok. 22 passed; 0 failed; 0 skipped; finished in 957.98µs (672.96µs CPU time)

Ran 12 tests for test/weirroll-helpers/SignedMathHelpers.t.sol:SignedMathHelpersTest
[PASS] test_Abs_Int256MinHandledViaUnchecked() (gas: 5738)
[PASS] test_Abs_PositiveAndNegative() (gas: 7042)
[PASS] test_AddOverflowReverts() (gas: 9083)
[PASS] test_AddSubRoundTripCrossingZero() (gas: 7550)
[PASS] test_Average_RoundsTowardsZero() (gas: 8059)
[PASS] test_Conditional_AppliesWhenTrue() (gas: 7032)
[PASS] test_Conditional_SkipsWhenFalse() (gas: 5818)
[PASS] test_DivByZeroReverts() (gas: 8956)
[PASS] test_MaxMin_AcrossSign() (gas: 8054)
[PASS] test_MulDivWithNegative() (gas: 7590)
[PASS] test_SubUnderflowReverts() (gas: 8676)
[PASS] test_Version() (gas: 5568)
Suite result: ok. 12 passed; 0 failed; 0 skipped; finished in 1.10ms (691.79µs CPU time)

Ran 5 tests for test/WeirollTestHelper.t.sol:WeirollTestHelperTest
[PASS] test_BuildApproveCommand() (gas: 95765)
[PASS] test_BuildTransferCommand() (gas: 97335)
[PASS] test_BuildWETHDepositCommand() (gas: 75595)
[PASS] test_BuildWETHWithdrawCommand() (gas: 60456)
[PASS] test_MockDEXSwapViaWeiroll() (gas: 143117)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 1.01ms (486.11µs CPU time)

Ran 22 tests for test/Router.Access.t.sol:RouterAccessTest
[PASS] test_Liquidator_CannotCallConfig() (gas: 28970)
[PASS] test_Liquidator_CannotSelfUpdate() (gas: 13864)
[PASS] test_Liquidator_Initial() (gas: 10500)
[PASS] test_Liquidator_OwnerCanUpdate() (gas: 26541)
[PASS] test_Ownable_InitialOwner() (gas: 13764)
[PASS] test_Ownable_NonOwner_CannotPause() (gas: 14633)
[PASS] test_Ownable_NonOwner_CannotSetLiquidator() (gas: 14216)
[PASS] test_Ownable_NonOwner_CannotSetPendingExecutor() (gas: 14681)
[PASS] test_Ownable_TransferTwoStep() (gas: 62161)
[PASS] test_Pause_BlocksSwap() (gas: 118946)
[PASS] test_Pause_BlocksSwapMulti() (gas: 64339)
[PASS] test_Pause_DoesNotBlockSweep() (gas: 77413)
[PASS] test_Pause_NonOwner_CannotPause() (gas: 23936)
[PASS] test_Pause_Unpause_Restores() (gas: 237844)
[PASS] test_SwapRouterFunds_ByOwner() (gas: 174351)
[PASS] test_SwapRouterFunds_ByStranger_Reverts() (gas: 31547)
[PASS] test_TransferRouterFunds_ArrayLengthMismatch_Reverts() (gas: 19315)
[PASS] test_TransferRouterFunds_ByLiquidator() (gas: 89025)
[PASS] test_TransferRouterFunds_ByOwner() (gas: 92741)
[PASS] test_TransferRouterFunds_ByStranger_Reverts() (gas: 21309)
[PASS] test_TransferRouterFunds_EmptyArrays_NoOp() (gas: 35994)
[PASS] test_TransferRouterFunds_ZeroDest_Reverts() (gas: 17342)
Suite result: ok. 22 passed; 0 failed; 0 skipped; finished in 4.00ms (3.51ms CPU time)
```

```
[PASS] test_SwapMultiPermit2_HappyPath_TwoInTwoOut() (gas: 583138)
[PASS] test_SwapMultiPermit2_RevertsOnNativeInputSlot() (gas: 31967)
[PASS] test_SwapPermit2_FeeOnTransferToken() (gas: 788688)
[PASS] test_SwapPermit2_HappyPath_ERC20toERC20() (gas: 306124)
[PASS] test_SwapPermit2_PausedReverts() (gas: 141458)
[PASS] test_SwapPermit2_RespectsAllSlippageAndFeeRules() (gas: 335049)
[PASS] test_SwapPermit2_RevertsOnExpiredDeadline() (gas: 126625)
[PASS] test_SwapPermit2_RevertsOnNativeInput() (gas: 24925)
[PASS] test_SwapPermit2_RevertsOnReusedNonce() (gas: 314622)
[PASS] test_SwapPermit2_RevertsOnTamperedAmount() (gas: 153471)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 4.12ms (6.84ms CPU time)

Ran 14 tests for test/Router.t.sol:RouterTest
[PASS] test_ExecutorRegistry_TwoStep() (gas: 876668)
[PASS] test_Revert_ETHValueMismatch_ERC20In() (gas: 118998)
[PASS] test_Revert_ETHValueMismatch_NativeIn() (gas: 36829)
[PASS] test_Revert_ExecutorBubblesUp_PreservesApproval() (gas: 245283)
[PASS] test_Revert_ExecutorNotSet() (gas: 2302451)
[PASS] test_Revert_InvalidSlippageBounds() (gas: 36533)
[PASS] test_Revert_Paused() (gas: 58844)
[PASS] test_Revert_SelfSwap() (gas: 36804)
[PASS] test_Revert_ZeroInputAmount() (gas: 36554)
[PASS] test_Revert_ZeroOutputMin() (gas: 36319)
[PASS] test_Revert_ZeroOutputQuote() (gas: 36112)
[PASS] test_SwapERC20ToERC20() (gas: 282741)
[PASS] test_SwapNativeETHIn() (gas: 117327)
[PASS] test_SwapNativeETHOut() (gas: 222190)
Suite result: ok. 14 passed; 0 failed; 0 skipped; finished in 5.22ms (11.37ms CPU time)

Ran 9 tests for test/Router.Adversarial.t.sol:RouterAdversarialTest
[PASS] test_FOT_InputToken() (gas: 645897)
[PASS] test_FOT_OutputToken() (gas: 770428)
[PASS] test_FalseReturningToken_Input_Reverts() (gas: 470782)
[PASS] test_FalseReturningToken_Output() (gas: 522897)
[PASS] test_NoReturnToken_Input() (gas: 505284)
[PASS] test_Rebasing_OutputNegativeRebase() (gas: 630462)
[PASS] test_Rebasing_OutputPositiveRebase() (gas: 660890)
[PASS] test_Reentrancy_ViaCallbackToken() (gas: 948966)
[PASS] test_Reentrancy_ViaETHReceive() (gas: 511811)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 5.55ms (2.64ms CPU time)

Ran 19 tests for test/ExecutionProxy.t.sol:ExecutionProxyTest
[PASS] testFuzz_ExecutePath_MintToRecipient(uint256,address) (runs: 256, : 71203, ~: 71119)
[PASS] test_Deploy() (gas: 5276)
[PASS] test_ExecutePath_DelegateCall_Reverts() (gas: 13704)
[PASS] test_ExecutePath_MintsTokensToRecipient() (gas: 69698)
[PASS] test_ExecutePath_MultiHopSwap() (gas: 234565)
[PASS] test_ExecutePath_SevenArgs_Succeeds() (gas: 262820)
[PASS] test_ExecutePath_SevenArgs_WithOuterValue() (gas: 269736)
[PASS] test_ExecutePath_WithApproveAndTransfer() (gas: 126041)
[PASS] test_ExecutePath_WithMockDEXSwap() (gas: 146763)
[PASS] test_ExecutePath_WithWETHUnwrap() (gas: 60301)
[PASS] test_ExecutePath_WithWETHWrap() (gas: 75638)
[PASS] test_Fallback_AcceptsETH() (gas: 17448)
[PASS] test_FlagData_ArbitraryCalldata_PassesVerbatim() (gas: 279539)
[PASS] test_FlagData_BackwardsCompat_VALUECALL_Unchanged() (gas: 75616)
[PASS] test_FlagData_EmptyCalldata_TriggersReceive() (gas: 137836)
[PASS] test_FlagData_NonVALUECALL_FlagIgnored() (gas: 66709)
[PASS] test_FlagData_ValueFromPriorCommandReturn() (gas: 193825)
[PASS] test_FlagData_ZeroValue_Succeeds() (gas: 104395)
[PASS] test_ReceiveETH() (gas: 17563)
Suite result: ok. 19 passed; 0 failed; 0 skipped; finished in 17.59ms (18.72ms CPU time)

Ran 18 tests for test/Router.Fees.t.sol:RouterFeesTest
[PASS] testFuzz_NoSilentShortPayment(uint256,uint256,uint256,uint16,uint16,bool) (runs: 256, : 292858, ~: 305487)
[PASS] test_NoFees_EmitsZeroFeeFields() (gas: 287200)
[PASS] test_PartnerFeeInput_CapExceeds_Reverts() (gas: 114494)
[PASS] test_PartnerFeeInput_Paid() (gas: 298954)
[PASS] test_PartnerFeeInput_StacksWithProtocolFee() (gas: 301866)
[PASS] test_PartnerFeeInput_ZeroRecipient_Reverts() (gas: 112101)
[PASS] test_PartnerFeeOutput_AfterSlippageCap() (gas: 299592)
[PASS] test_PartnerFeeOutput_Paid() (gas: 301252)
[PASS] test_PartnerFeeOutput_PushesBelowOutputMin_Reverts() (gas: 272873)
```

```
[PASS] test_PositiveSlippage_CapturedWhenFlagOff() (gas: 278151)
[PASS] test_PositiveSlippage_NoSurplus_Noop() (gas: 278122)
[PASS] test_PositiveSlippage_PassThroughWhenFlagOn() (gas: 277632)
[PASS] test_ProtocolFee_AccumulatesInRouter() (gas: 429546)
[PASS] test_ProtocolFee_AtCap() (gas: 275761)
[PASS] test_ProtocolFee_CapExceeds_Reverts() (gas: 112456)
[PASS] test_ProtocolFee_Charged() (gas: 278318)
[PASS] test_ProtocolFee_ZeroBps() (gas: 276224)
Suite result: ok. 18 passed; 0 failed; 0 skipped; finished in 69.02ms (73.47ms CPU time)

Ran 12 tests for test/Router.MultiSwap.t.sol:RouterMultiSwapTest
[PASS] test_MultiSwap_ArrayLengthMismatch_Reverts() (gas: 33399)
[PASS] test_MultiSwap_DuplicateInput_Reverts() (gas: 38396)
[PASS] test_MultiSwap_DuplicateOutput_Reverts() (gas: 39690)
[PASS] test_MultiSwap_EmptyInputs_Reverts() (gas: 30668)
[PASS] test_MultiSwap_EmptyOutputs_Reverts() (gas: 30963)
[PASS] test_MultiSwap_InputOutputIntersection_Reverts() (gas: 37043)
[PASS] test_MultiSwap_PerOutputSlippage() (gas: 257780)
[PASS] test_MultiSwap_ProRataPartnerFee_OnOutput() (gas: 587760)
[PASS] test_MultiSwap_ProRataProtocolFee() (gas: 548398)
[PASS] test_MultiSwap_TwoInTwoOut() (gas: 554441)
[PASS] test_MultiSwap_WithNativeETH_AsOneInput() (gas: 306728)
[PASS] test_MultiSwap_ZeroAmountInput_Reverts() (gas: 37632)
Suite result: ok. 12 passed; 0 failed; 0 skipped; finished in 1.05s (5.72ms CPU time)

Ran 3 tests for test/Router.Invariant.t.sol:RouterInvariantTest
[PASS] invariant_AccountingConservation() (runs: 256, calls: 3840, reverts: 468)

-----+-----+-----+-----+-----+
| Contract      | Selector      | Calls | Reverts | Discards |
+-----+-----+-----+-----+-----+
| RouterHandler | swapMultiRandom | 1933  | 136    | 0        |
+-----+-----+-----+-----+-----+
| RouterHandler | swapRandom      | 1907  | 332    | 0        |
+-----+-----+-----+-----+-----+

[PASS] invariant_NoDustLost() (runs: 256, calls: 3840, reverts: 504)

-----+-----+-----+-----+-----+
| Contract      | Selector      | Calls | Reverts | Discards |
+-----+-----+-----+-----+-----+
| RouterHandler | swapMultiRandom | 1913  | 137    | 0        |
+-----+-----+-----+-----+-----+
| RouterHandler | swapRandom      | 1927  | 367    | 0        |
+-----+-----+-----+-----+-----+

[PASS] invariant_NoResidualUserInput() (runs: 256, calls: 3840, reverts: 451)

-----+-----+-----+-----+-----+
| Contract      | Selector      | Calls | Reverts | Discards |
+-----+-----+-----+-----+-----+
| RouterHandler | swapMultiRandom | 1984  | 112    | 0        |
+-----+-----+-----+-----+-----+
| RouterHandler | swapRandom      | 1856  | 339    | 0        |
+-----+-----+-----+-----+-----+

Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 1.19s (3.06s CPU time)
Ran 12 test suites in 1.19s (2.35s CPU time): 146 tests passed, 1 failed, 0 skipped (147 total tests)

Failing tests:
Encountered 1 failing test in test/weiroll-helpers/UniswapV4SwapHelpers.fork.t.sol:UniswapV4SwapHelpersForkTest
[FAIL: EvmError: Revert] setUp() (gas: 0)
Encountered a total of 1 failing tests, 146 tests succeeded
```

8.2 Automated Tools

8.2.1 AuditAgent

The AuditAgent is an AI-powered smart contract auditing tool that analyses code, detects vulnerabilities, and provides actionable fixes. It accelerates the security analysis process, complementing human expertise with advanced AI models to deliver efficient and comprehensive smart contract audits. Available at <https://app.auditagent.nethermind.io>.

9 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our cryptography Research team conducts cutting-edge internal research and collaborates closely with external partners on cryptographic protocols, consensus design, succinct arguments and folding schemes, elliptic curve-based STARK protocols, post-quantum security and zero-knowledge proofs (ZKPs). Our research has led to influential contributions, including Zinc (Crypto '25), Mova, FLI (Asiacrypt '24), and foundational results in Fiat-Shamir security and STARK proof batching. Complementing this theoretical work, our engineering expertise is demonstrated through implementations such as the Latticefold aggregation scheme, the Labrador proof system, zkvm-benchmarks, and Plonk Verifier in Cairo. This combined strength in theory and engineering enables us to deliver cutting-edge cryptographic solutions to partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to [Nethermind](#) in order that [Nethermind](#) could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. [Nethermind](#) has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, [Nethermind](#) disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. [Nethermind](#) does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and [Nethermind](#) will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.